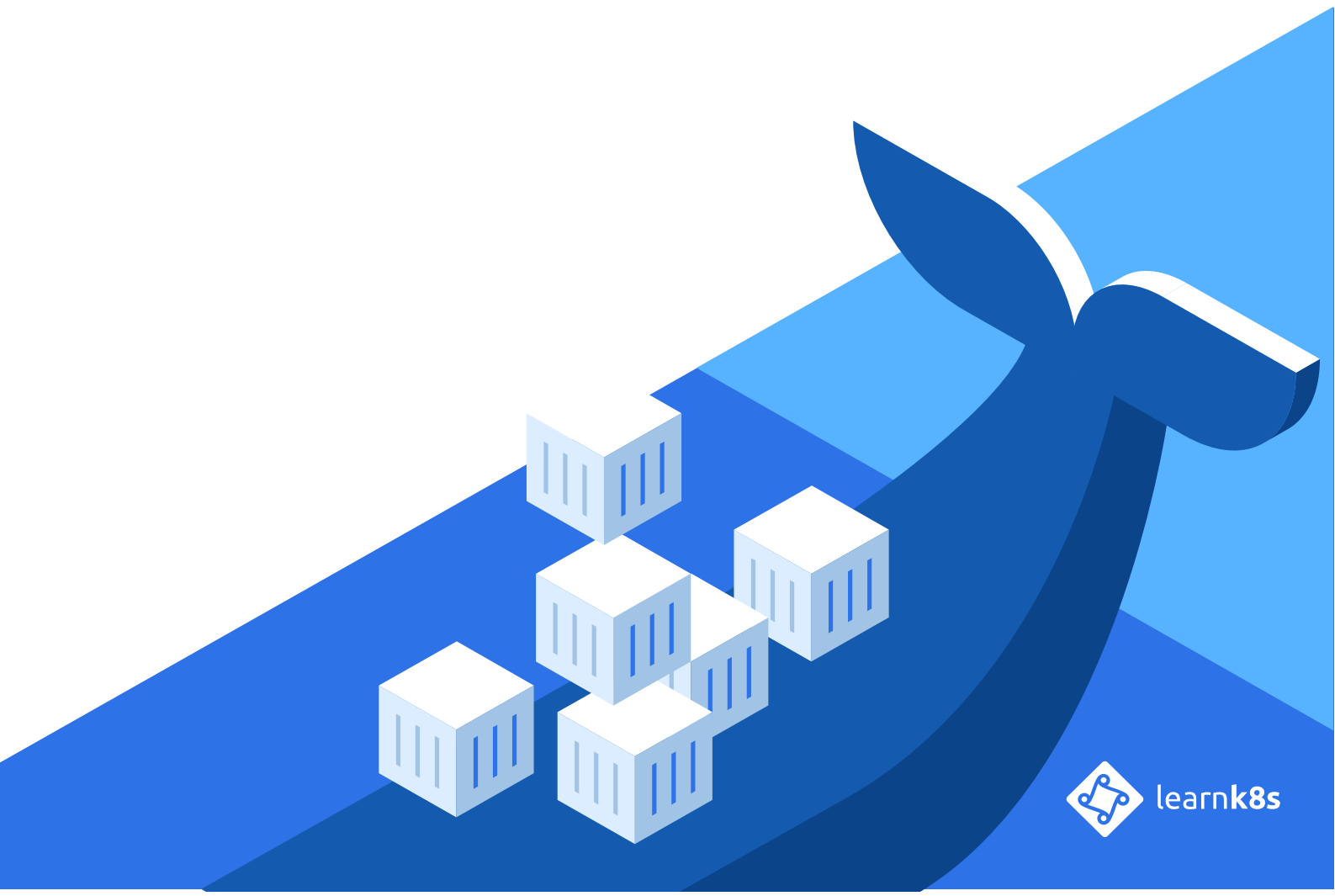
LINUX CONTAINERS AND KUBERNETES



learnk8s

Table of contents

1. Why containers? 6

Managing infrastructure at scale 9

Underutilised compute resources 12

Poor resource allocation 16

Proliferation of languages and frameworks 22

Containers in the shipping industry 25

Linux containers 25

What are containers made of? 28

2. Containers and Docker 32

Running containers 36

Building Docker images 37

Managing containers 39

Sharing the file system 40

COPYing files 41

Forwarding ports 43

Managing configuration 46

Attaching to running containers 47

3. Installing Docker 🐋 49

Windows 10 50

macOS 55

Testing your Docker installation 56

Ubuntu 57

4. Getting started with Docker 60

Running Docker containers 61

Docker registries 62

Running commands inside containers 63

Start, stop, list, kill 65

Terminology 69

Recap 69

5. Creating images 71

Your first Dockerfile 73

Building the image 74

Recap 77

6. Managing state 79

Sharing the file system 80

Exploring bidirectional sync 81

Including files in the container image 82

Recap 85

7. Networking 86

Connecting to the container 87

Binding container ports to the host 89

Recap 90

8. Packaging an application 92

Hello World, Node.js 93

Creating an image 94

Injecting environment variables 97

Recap 98

9. Troubleshooting 100

Attaching to a running container 101

Common challenges when attaching to containers 103

Troubleshooting build errors 104

Recap 107

Chapter 1

Why containers?

During the last few years, the industry has experienced a shift toward developing smaller and more focused applications.

It doesn't come as a surprise that more and more companies are breaking their massive and static monoliths into a set of decoupled and independent components.

And rightly so.

Services that are tiny in size are:

- **quicker to deploy** because you create and release them in smaller chunks
- **easier to iterate on**, since adding features happens independently
- **resilient** — the overall service can still function despite one of the components not being available

> Please bear in mind that developing smaller components have disadvantages too. As an example, managing dependencies and versioning is complicated when you have several components interacting with each other.

But how does that cultural shift impact the infrastructure?

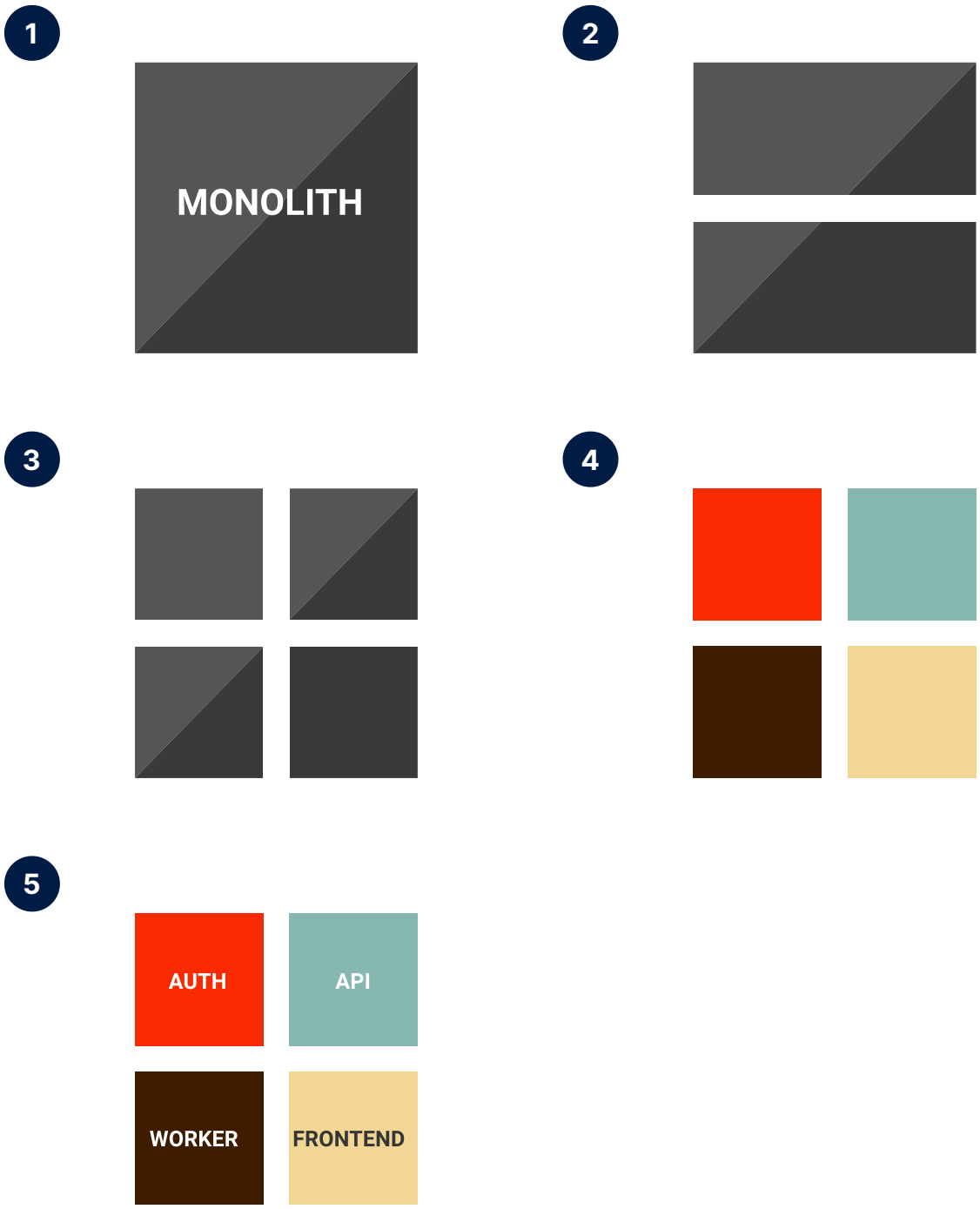


Fig. 1 Applications packaged as a single unit as usually referred to as monoliths.

Fig. 2 You might have started developing applications as a single units and then decided to break them down into smaller components.

Fig. 3 Or you may have inherited a large application from someone else and decided to separate several concerns into different modules.

Fig. 4 At the end of the process you end up with a collection of smaller, more focussed apps.

Fig. 5 From a single deployable unit to four. Is the change impacting how you deploy the applications?

Managing infrastructure at scale

Developing services out of smaller components, in fact, introduces a different challenge.

In the past, you might have deployed applications in virtual machines.

When for every application, you can refactor the same app in a collection of four components, **you have at least four times more apps to develop, package and release.**

You also have three more virtual machines to manage.

It's not uncommon for a small service to be made out of a dozen components such as a front-end app, a backend API, an authorisation server, or an admin portal.

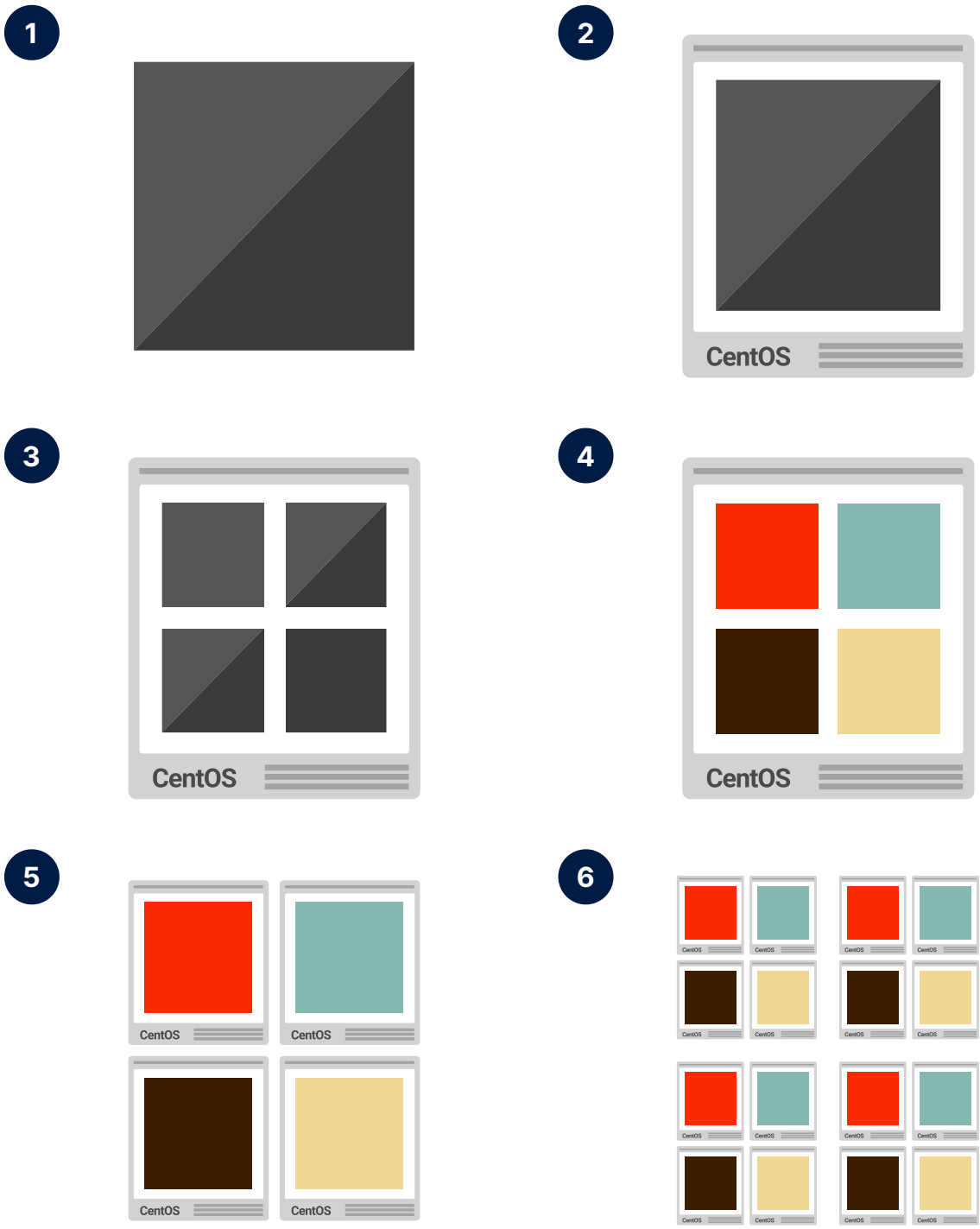


Fig. 1 When you have a single large application, you can deploy it inside a virtual machine.

Fig. 2 When you have a single large application, you can deploy it inside a virtual machine.

Fig. 3 As soon as you decide to break the application into smaller components, you should find a way to isolate those components so that failures as segregated and scaling happens independently.

Fig. 4 As soon as you decide to break the application into smaller components, you should find a way to isolate those components so that failures as segregated and scaling happens independently.

Fig. 5 You might be tempted to wrap each app into a separate virtual machine.

Fig. 6 When you have several components, you end up using more virtual machines than you did previously with a single unit.

You might wonder what's the issue with having several virtual machines.

Underutilised compute resources

Each virtual machine comes with an operating system that consumes part of the memory and CPU resources allocated to it.

When you create a t2.micro Elastic Compute Cloud instance in Amazon with 1GB of memory and 1 vCPU, you end up using 700MB in memory, and 80% of the CPU after you remove the overhead of the operating system.

Similarly, when you request for four larger instances such as a t2.xlarge with 4 vCPU and 16 GiB of memory each, you will end up wasting about 3 vCPU and 1.2GiB of memory in total.

The overhead adds up to about 2 to 3%.

Not a lot.

When you refactor the four applications in a collection of 4 smaller components, you have 16 virtual machines deployed in your infrastructure.

Each virtual machine has an operating system that wastes 300MiB of memory and 0.2 vCPU.

In total, you waste 4.8GiB of memory (300MiB times 16 apps) and 3.2 vCPU (0.2 vCPU times 16 apps).

That's a 6 to 10% of resources you pay for, but can't use.

If your cloud bill is $1000 per month, you are wasting $100

per month in running operating systems.
Let's have a look at an example.

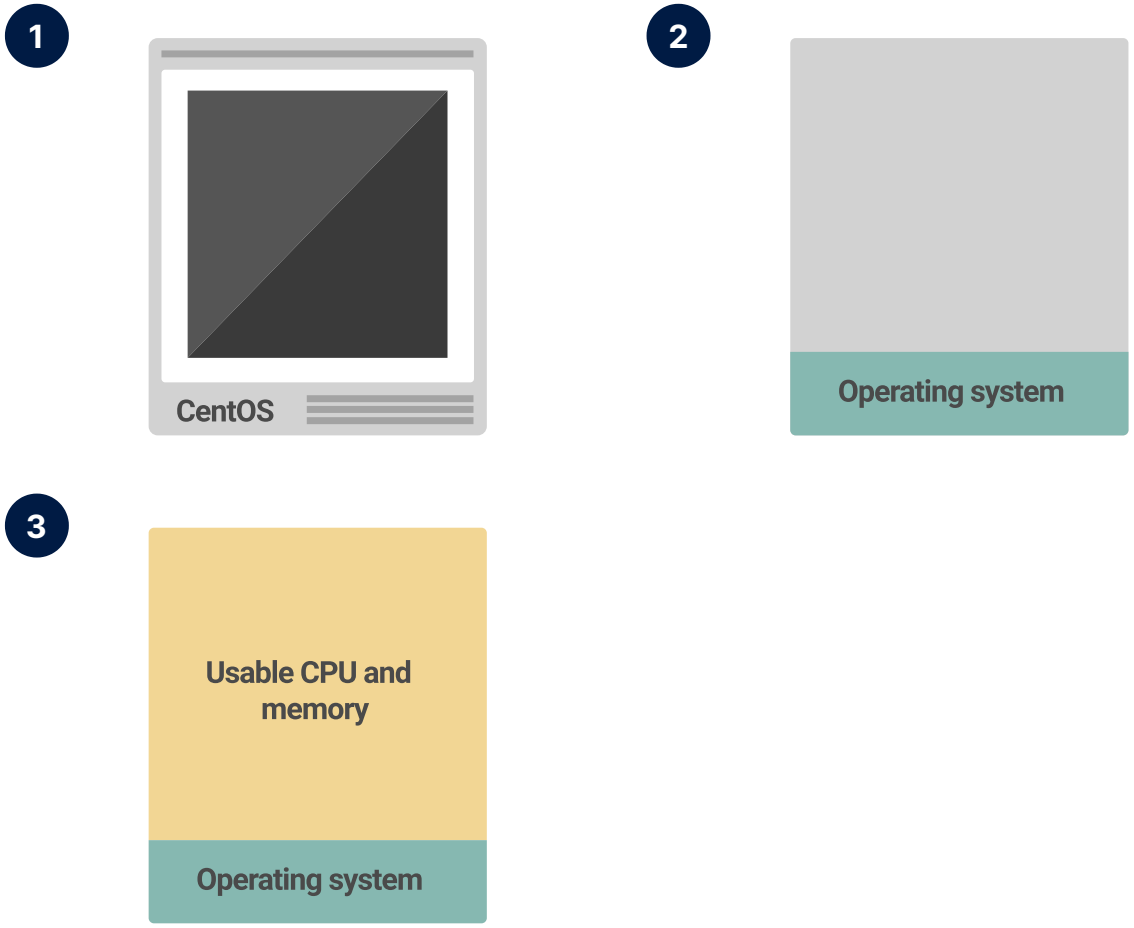


Fig. 1 Consider the following virtual machine.

Fig. 2 Part of the resources of that virtual machine are used for the operating system.

Fig. 3 The application can use the rest of the CPU and memory.

This is for a single virtual machine.

But what happens when you have a few of them?

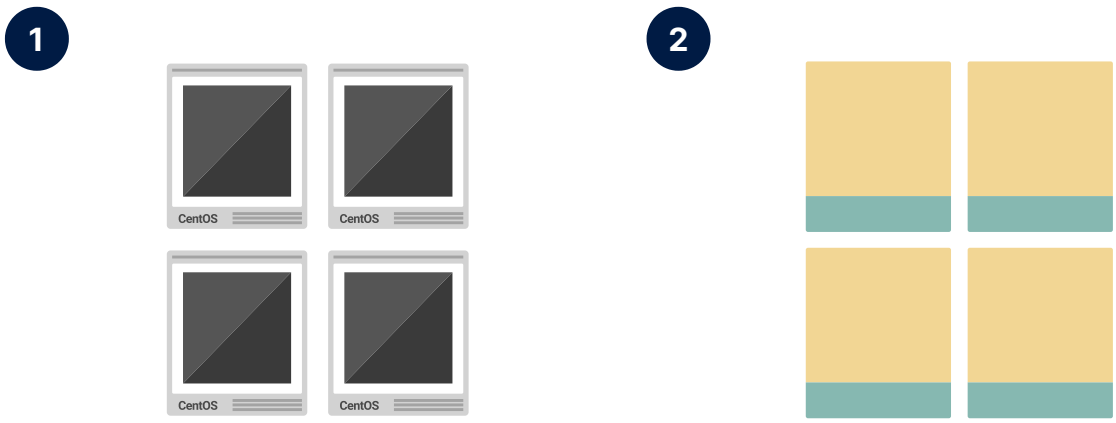


Fig. 1 When you develop fewer and larger apps, you have a limited number of virtual machines deployed in your infrastructure. They also tend to use compute resources with higher specs.

Fig. 2 Few large virtual machines with operating systems don't have a significative impact on the resource utilisation. The overhead from running the operating system is between 2 and 3%.

And what about microservices?

What happens when you have plenty of smaller virtual machines?

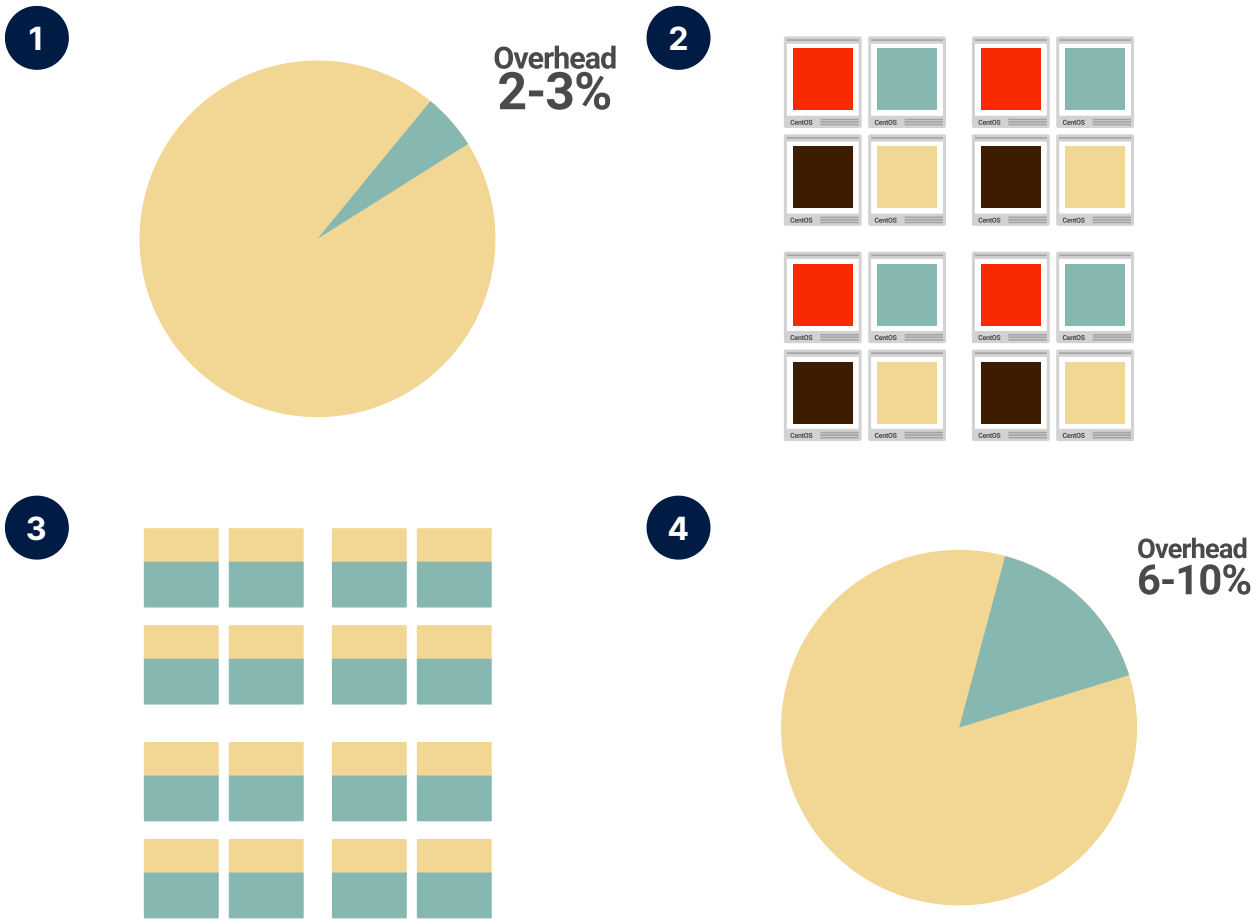


Fig. 1 Now imagine having the four large units broken down into a collection of smaller components. Each component has its virtual machine.

Fig. 2 Each virtual machine has an operating system. Where previously you had four, now you have 16.

Fig. 3 Having several virtual machines with generally more modest specs means that the operating system uses (in proportion) more resources.

Fig. 4 The overhead is more significative too. Depending on your setup it's not uncommon to waste 10% of resources in running operating systems.

However, the operating system overhead is only part of the issue.

Poor resource allocation

You have probably realised that when you break your service into smaller components, each of them comes with different resource requirements.

Some components such as data processing and data mining applications are CPU intensive.

Others, such as servers for real-time applications might use more memory than CPU.

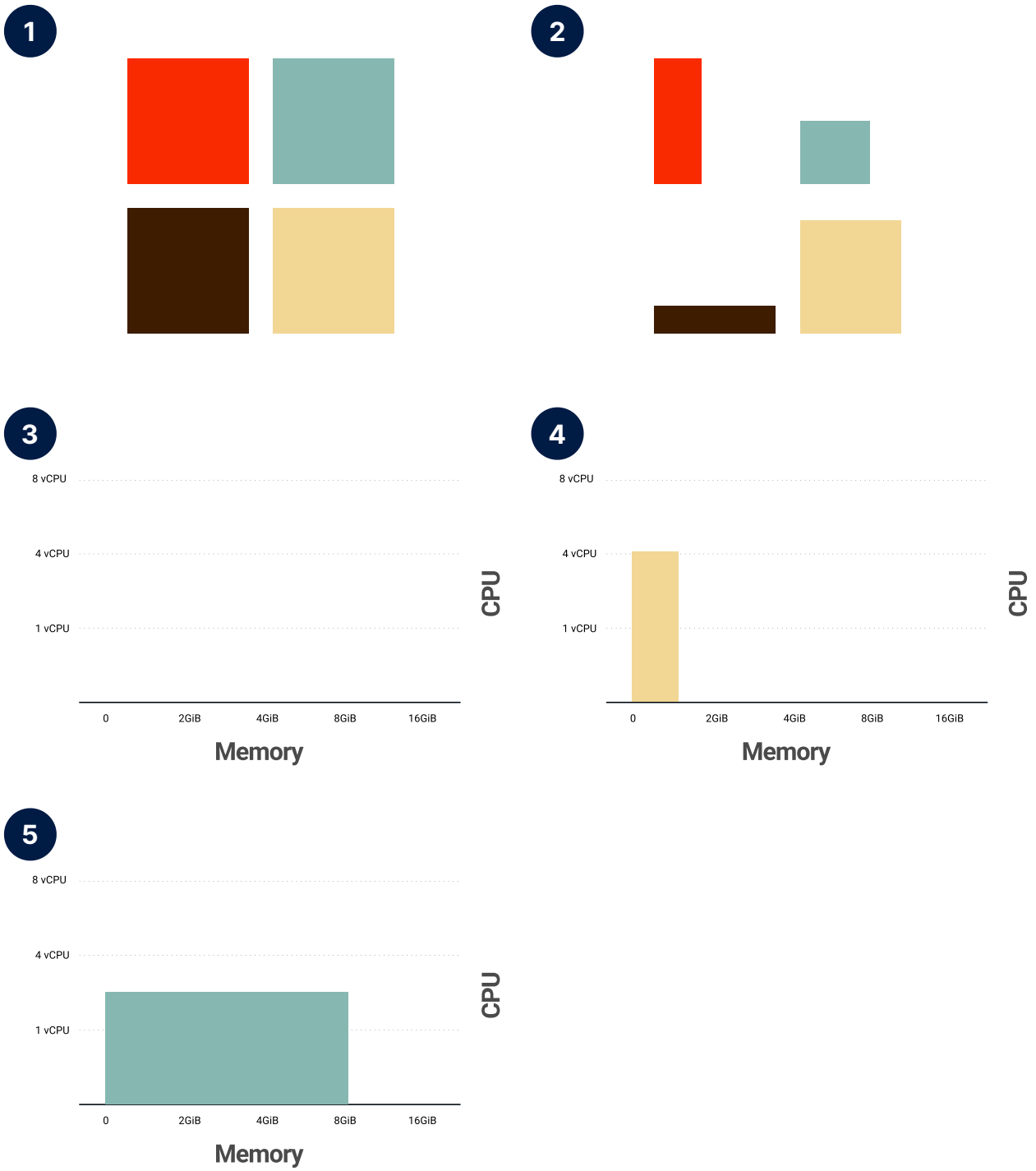


Fig. 1 When you develop apps as a collection of smaller components, you realised that no two apps are alike.

Fig. 2 They all look different. Some of them are CPU intensive other require more memory. And you might have apps requiring specific hardware such as GPUs.

Fig. 3 You can imagine being able to measure those apps.

Fig. 4 Some of them could be more CPU heavy than others.

Fig. 5 Or you could have components that use similar CPU resources but a lot more memory.

Ideally, you should strive to use the right virtual machine that fits for your component's requirements.

If you have an app that uses 800MiB of memory, you don't want to use a virtual machine that has 2GiB.

You're probably fine with one that has 1GiB.

In practice, this is not always the case.

It's much easier to select a single virtual machine that is good enough in 80% of the cases and use it all the times.

The result?

You waste hundreds of megabytes of RAM and plenty of CPU cycles in underutilised hardware.

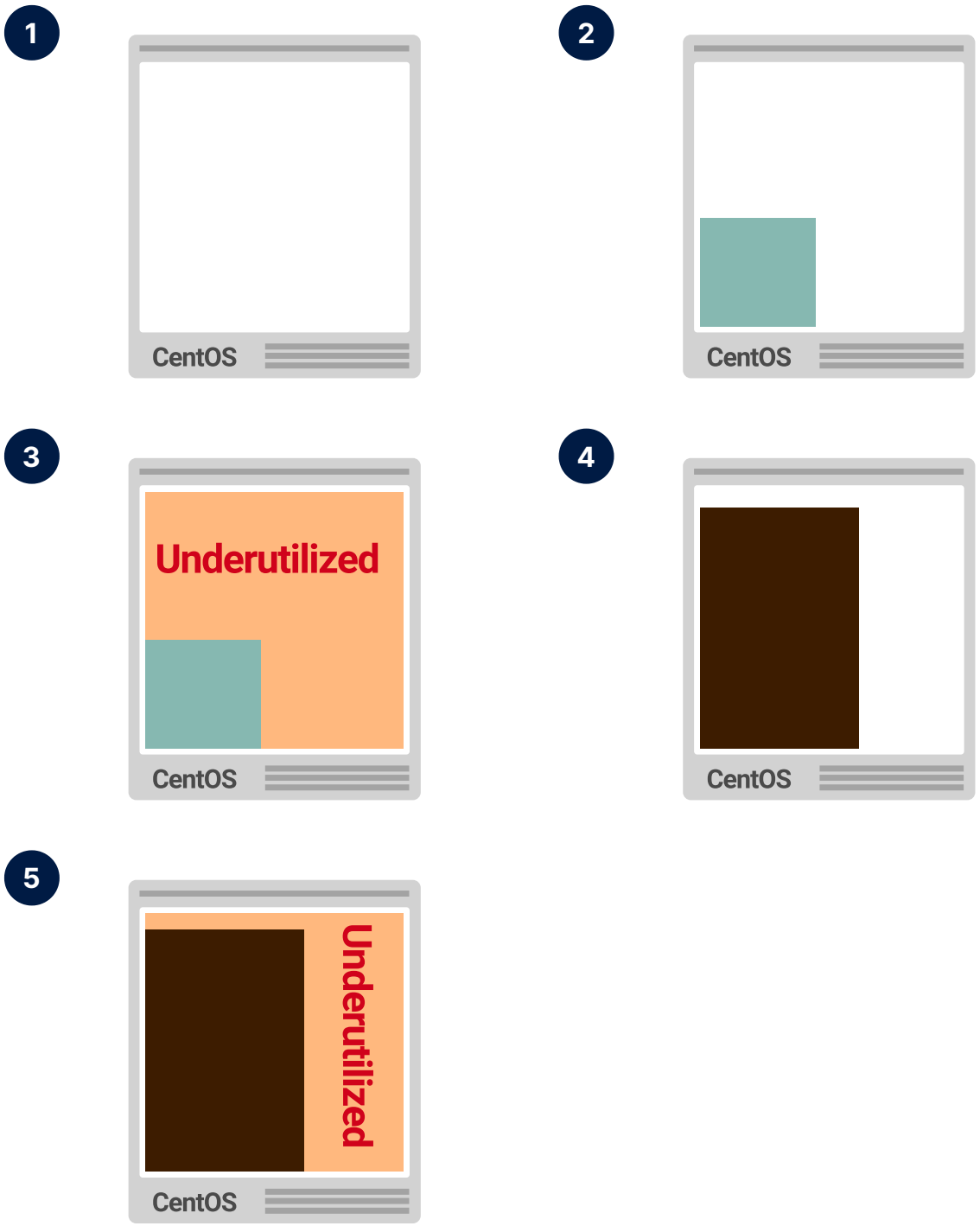


Fig. 1 Imagine having a virtual machine with 4 GiB of memory and 4 vCPU.

Fig. 2 You can deploy an application that uses only 1GiB of memory and 1 vCPU, but you're wasting 3/4 of the resources.

Fig. 3 You can deploy an application that uses only 1GiB of memory and 1 vCPU, but you're wasting 3/4 of the resources.

Fig. 4 It's common to use the same virtual machine's spec for all apps. Sometime you might be lucky and minimise the waste in resources.

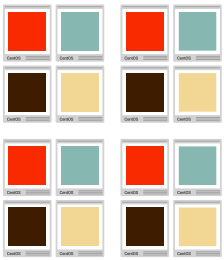
Fig. 5 It's common to use the same virtual machine's spec for all apps. Sometime you might be lucky and minimise the waste in resources.

Some companies utilise as little as 10% of their allocated resources.

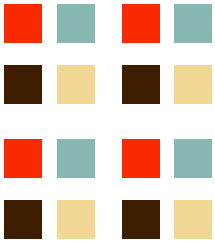
Can you imagine allocating hundreds of servers, but effectively using entirely only a dozen of them?

What you need is something to break free from the fixed resource allocation of a virtual machine and regain the resource you don't use.

1



2



3

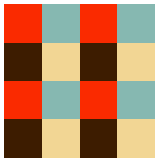


Fig. 1 What you want is to get rid of those wrappers around your applications. You don't want to run virtual machines; you only want to run your apps.

Fig. 2 What you want is to get rid of those wrappers around your applications. You don't want to run virtual machines; you only want to run your apps.

Fig. 3 Ideally, if you didn't have to deal with the wrapper, you could pack all of your apps tightly together and save a lot of space.

But why are those components so different in specs anyway?!

Why can't you have only lovely squares?

Proliferation of languages and frameworks

When you develop smaller and isolated components, you can pick "the right tool for the job" to build your application.

Java and Spring boot for the backend.

Node.js and Express for the front-end.

Rails and Resque for processing background tasks.

You name it.

You can imagine the proliferation of libraries and languages at scale.

And even if you try to force only a few libraries and frameworks, it might still not be enough.

Two applications sharing the same JVM runtime might rely on a different set of dependencies and libraries and have different resource requirements.

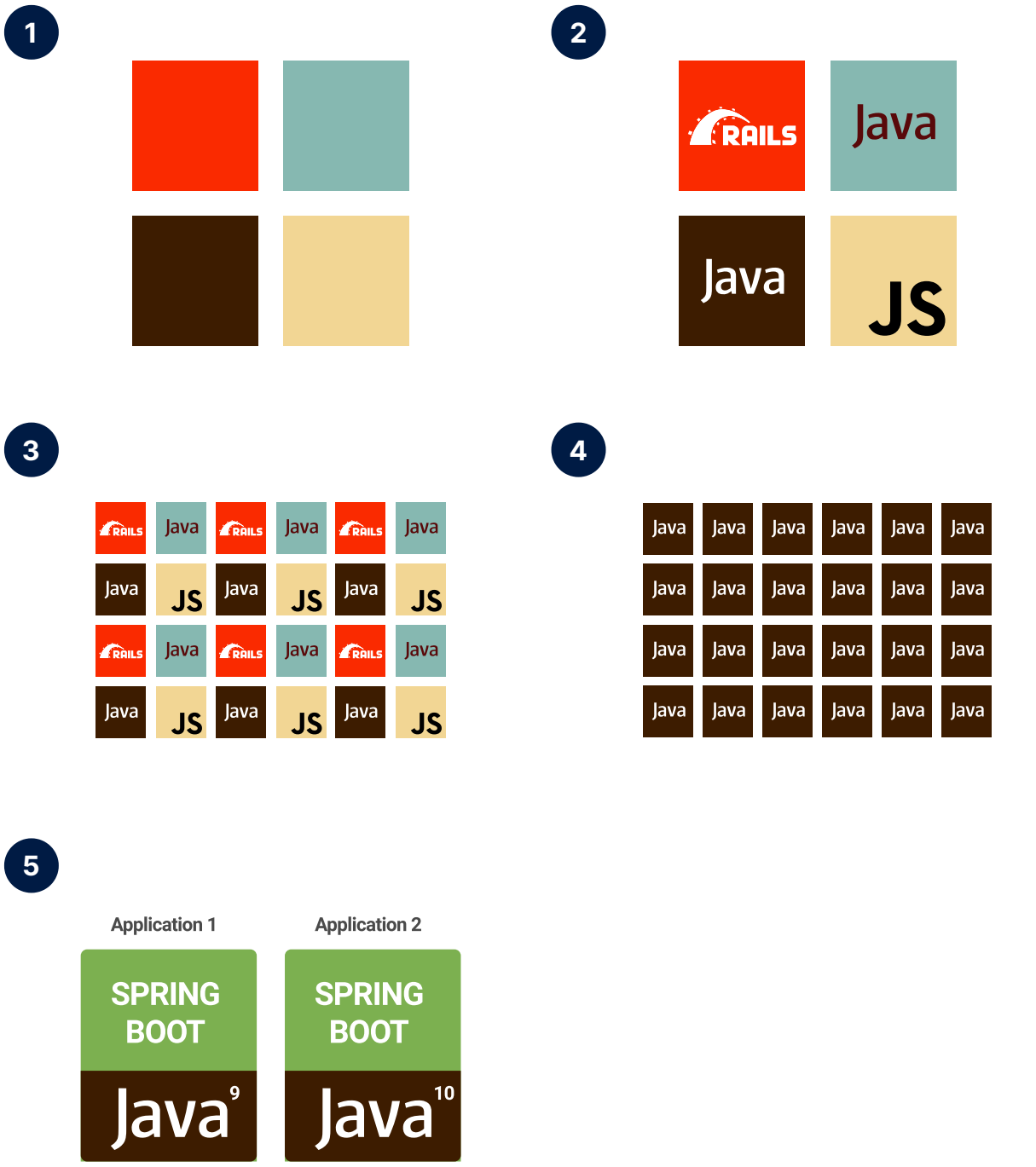


Fig. 1 Imagine developing a service as a collection of smaller components.

Fig. 2 You could pick a different language for each component. You're not limited to use only one like in the case of the monolith.

Fig. 3 If you have hundreds of smaller components, you could have several languages, libraries as well as different versions of the same languages.

Fig. 4 And even if you standardise on a single language, you still face the challenge of having different versions of the runtime of dependent libraries.

Fig. 5 Take for example a web app built using Spring Boot and a service broker that uses ZeroMQ with Spring Boot. They're both running on the same version of the JVM, but one requires a binary to be installed on the host.

Traditionally the environment and the application were developed and packaged separately.

You had to rely on tools such as Puppet or Ansible to configure your virtual machine with the right dependencies.

The approach is still valid, but it doesn't scale when you have hundreds or thousands of components to package.

Ideally, **you should keep all of the dependencies alongside your component.**

You shouldn't treat the infrastructure as an afterthought.

But how do you keep dependencies and applications together?

Containers in the shipping industry

Not long ago, the shipping industry had a similar challenge.
How do you ship goods in cargo around the world?
Do you load individual items of all shapes and sizes?
How do you know all the items that belong to the same importer?
Manually keeping track of things is hard.
So the shipping industry turned to containers.
A container is more convenient to load and deliver because it has a standard size.
It would be great if you could use a container for your components too.

Linux containers

You could pack your applications and all of the dependencies in a standard package to be deployed.

Enter Linux containers.

A Linux container is like a cargo container, but it encapsulates all files, binaries, and libraries necessary to run your process.

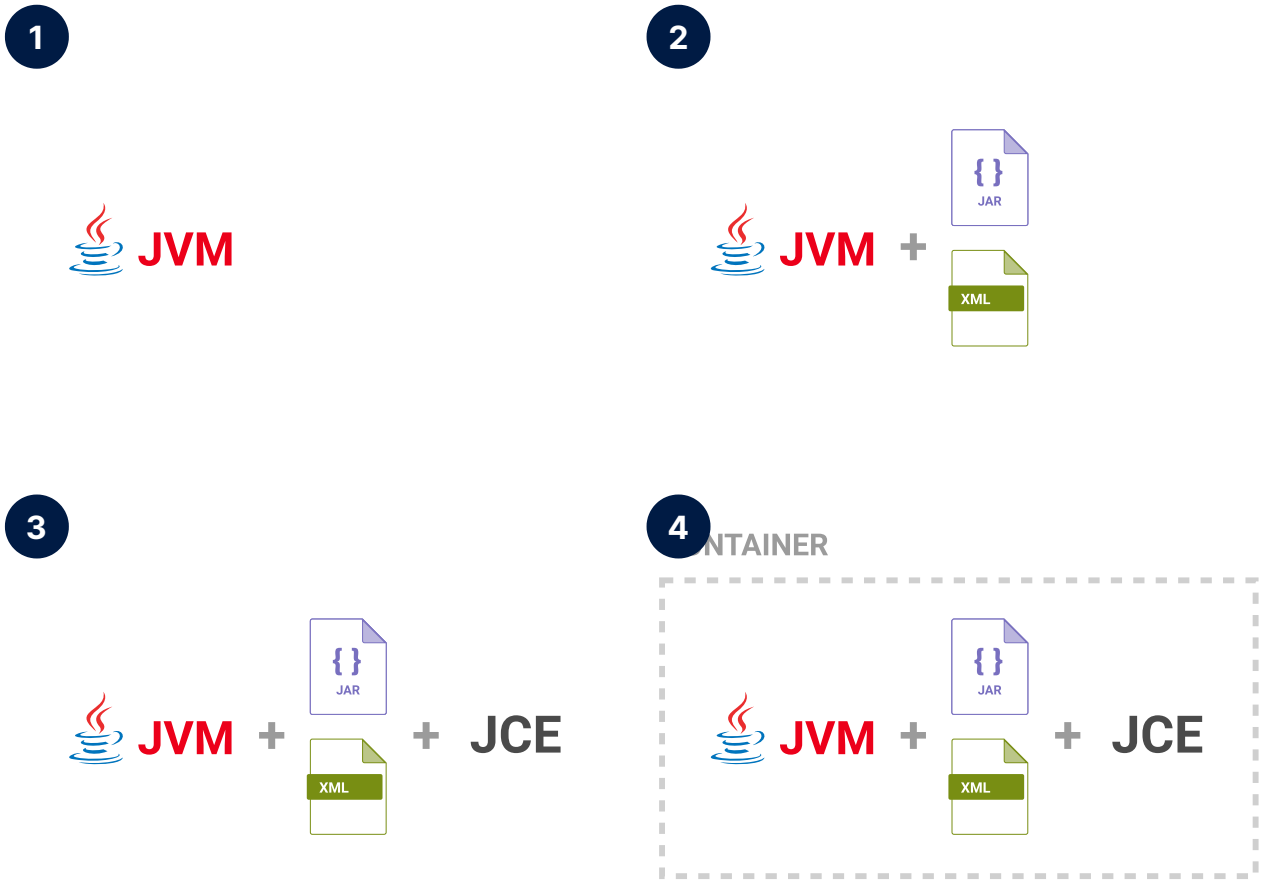


Fig. 1 Your application might have dependencies such as a runtime (e.g. JVM), files such as configuration and artefacts, and external dependencies such as ImageMagick or the Java Cryptographic Extension (JCE).

Fig. 2 Your application might have dependencies such as a runtime (e.g. JVM), files such as configuration and artefacts, and external dependencies such as ImageMagick or the Java Cryptographic Extension (JCE).

Fig. 3 Your application might have dependencies such as a runtime (e.g. JVM), files such as configuration and artefacts, and external dependencies such as ImageMagick or the Java Cryptographic Extension (JCE).

Fig. 4 Instead of leaving setting the environments to Puppet or Ansible, you could package all of your assets together in a container.

You might be wondering if this doesn't sound a lot like a virtual machine with an integrated configuration management tool.

As you realised early on:

- virtual machines are virtualised environments with an operating system that consumes resources
- when you launch and allocate memory and CPU for a virtual machine, you can't claim unused resources back
- virtual machines are like a computer. They are slow to

- start, and they emulate hardware

On the other hand:

- **Containers isolate processes**. When you start a container, you're just launching a process on your computer
- The process will **consume only the memory and CPU that it needs**
- **Processes are quick to start**, and there's no need for an operating system

Processes encapsulated in containers, can't see each other.
Like if they were packaged in virtual machines.
But the isolation provided by containers is different from virtual machines.

What are containers made of?

Containers are constructed from two feature in the Linux kernel: **control groups** and **namespaces**.
Control groups are a convenient way to limit the CPU or memory that a particular process can use.
As an example, you could say that your component should use only 2GB of memory and one of your four CPU cores.
Namespaces, on the other hand, are in charge of isolating

the process and limiting what it can see.

The component can only see the network packets that are directly related to it.

It won't be able to see all of the network packets flowing through the network adapter.

Control groups and namespaces are low-level primitives.

With time developers created more and more layers of abstractions to make it easier to control those kernel features.

One of the first abstractions was LXC, but the real deal was Docker that was released in 2013.

With containers:

- **you only pay for the resources that you use**. No need for an extra operating system for each component
- **you can package and deploy applications with a standard interface**. You don't have to worry what binary, language or framework is encapsulated in the container
- **multiple isolated processes are running on the same host**

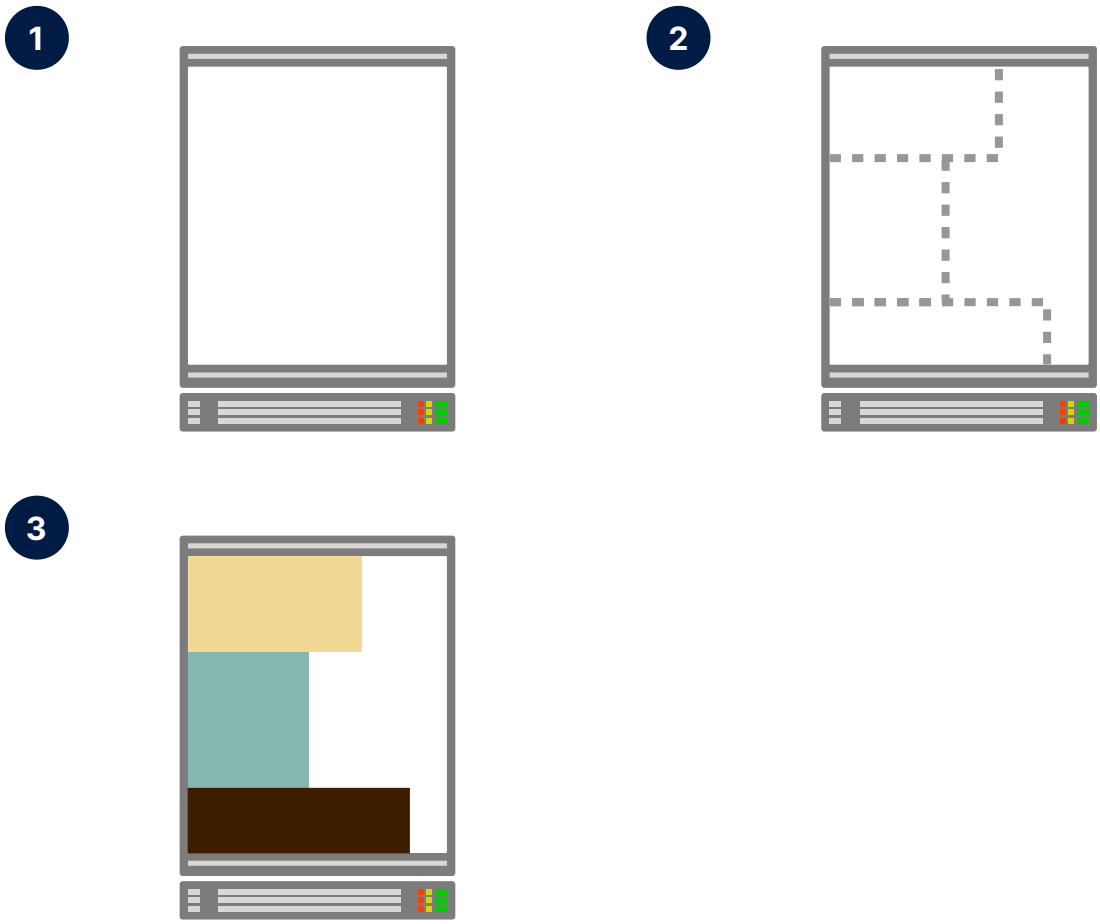


Fig. 1 Imagine having a server or a virtual machine.

Fig. 2 With Linux containers, you can define discrete slots for resources such as memory and CPU and have your processes use and see only those resources.

Fig. 3 With Linux containers, you can define discrete slots for resources such as memory and CPU and have your processes use and see only those resources.

You'll see later on how you can leverage Kubernetes to pack

your infrastructure efficiently.

Chapter 2

Containers and Docker

Docker uses a client-server architecture.

The Docker client talks to the Docker daemon using the REST API, over UNIX sockets or a network interface.

When you want to run a Docker container, the Docker client sends the request to the Docker daemon, and the container runs wherever the daemon is installed.

The Docker client and daemon can run on the same computer, or you can connect a Docker client to a remote Docker daemon.

When you connect to a remote daemon, the containers are created remotely.

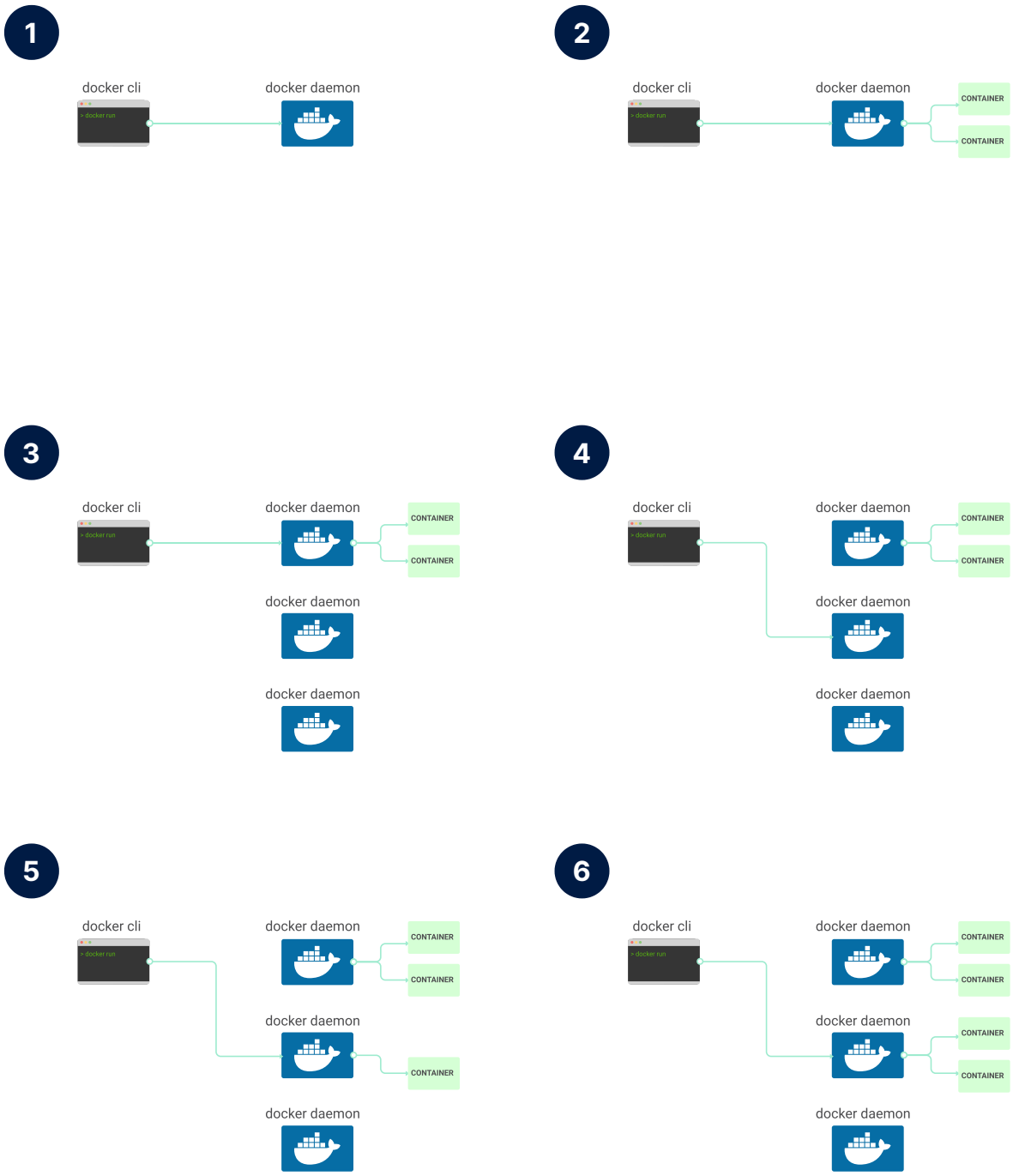


Fig. 1 You can connect to a Docker using the REST API, over UNIX sockets or a network interface.

Fig. 2 The daemon creates the containers.

Fig. 3 But you could connect to other Docker daemons. Daemon could be located remotely.

Fig. 4 You can change the settings and have your client connecting to a different daemon.

Fig. 5 You aren't able to inspect the previous container since the previous Docker daemon created them. You can create more containers, though.

Fig. 6 You could connect to more remote Docker daemons and repeat the process.

Connecting to a remote Docker daemon is such a typical operation that you might not even notice about it.

If you're using Windows or macOS to run your containers you might wonder how Docker works.

Aren't Linux containers (and Docker) ***a Linux feature?***

How can you run Docker containers on Windows or macOS?

As you probably already guessed, Docker for Windows and macOS run inside a virtual machine with Linux.

The Docker client is connected to the Docker daemon inside the virtual machine.

All the containers that you created live inside the Linux

virtual machine.

> Please note that if you're running Docker for Linux, the Docker daemon can run natively and doesn't use any extra virtual machine.

Running containers

You can start a Docker container with:

```bash
$ docker run -ti ubuntu bash
```

You can break down the command in the following parts:

- `docker run` is the command that you use to start a container
- `-ti` means that you wish to run an interactive session
- `ubuntu` is the name of the image
- `bash` is the process you wish to run inside the container

Docker containers are made from Docker images.
Images are a collection of files organised as layers.

You could think about Docker images as archives such as a zip or tar archive.

When you launch a container, you run a process such as `bash` with all the files that you'd generally expect inside an Ubuntu Linux distro.

The process is isolated, and everything feels and looks like it's running in an Ubuntu.

The reality is that it is running as a process on your computer (or inside a virtual machine if you have Docker for macOS or Windows).

Images are useful, and perhaps you could create your own?

Building Docker images

You learned that Docker images are archives.

Also, you probably realised that from a single image you could create several containers.

You can keep typing `docker run -ti ubuntu bash` and more `bash` processes are spawned as a consequence.

You could think about Docker images as classes in Java.

They don't exist until you instantiate them.

When you create an instance of a Docker image, that's called a container.

You can create your personal Docker images by writing a list of instructions.

That list is like a recipe.

You can send that recipe to the daemon, and Docker executes the instructions one by one and creates the image.

The instructions are stored in a file called `Dockerfile`:

Dockerfile

```
FROM ubuntu
RUN apt-get update
RUN apt-get -y install figlet
```

You can see two commands: `FROM` and `RUN`.

- `FROM` is used to inherit from a base image. When the image is created, the Ubuntu image with all its files is used as the base
- `RUN` execute a command inside the image

In the example above, you start from a Ubuntu image and install the `figlet` binary — a program to draw ASCII art.

> There is a list of all available commands on the official Docker documentation.

Once you create a `Dockerfile` locally, you can ask the Docker daemon to build the image with:

```bash
$ docker build -t my-image .
```

The command reads as follow:

- `docker build` is the command to build an image
- `-t my-image` is the name you wish to give to the image
- `.` is the directory where the `Dockerfile` is located.

It's common practice to have the Dockerfile in the root of the project (hence the `.` dot).

Managing containers

You can launch containers with `docker run`*, but how do you know they are running?*
You can list all running containers with:

```bash
$ docker ps
```

If you wish to stop a container you can use either its name or id like this:

```bash
$ docker stop <name or id>
```

The `docker stop` command waits for the process to shut down gracefully.

If you prefer not to wait you can force the process to exit with:

```bash
$ docker kill <name or id>
```

Once the container is stopped, it doesn't appear in the `docker ps`.

However, you can still see the stopped container with:

```bash
$ docker ps -a
```

Sharing the file system

There are times when you want your containers to share files with your computer.

Perhaps you want to edit a python script locally with your IDE and have the container run it.

When you wish to share files with a container you can mount that folder inside the container with:

```bash
$ docker run -ti -v /my-computer:/app ubuntu bash
```

You're already familiar with the `docker run` command.

The `-v` flag is a shorthand for `--volume` and is designed to mount the local folder `/mycomputer` inside the container as `/app`.

Please bear in mind that you can only share files at runtime after the image was created.

So what should you do when you want to package assets such as Java wars inside the container?

COPYing files

You can bundle files, artefacts and assets when you build the image by adding an extra instruction to your `Dockerfile`.

The `COPY` command takes two arguments:

1. the location of the folder or file in your computer
2. the location inside the container where the file should be saved

Dockerfile

```
FROM ubuntu
COPY ./mycomputer /app-container
```

The `COPY` command is similar to the `-v` (volume) flag you learnt earlier.

When should you use one or the other?

You should use the `COPY` command every time you wish to bundle your artefacts with the image.

When you start the container, the files are already there, no need to mount them.

You should use the `-v` (or `--volume`) flag when you want to share files with the container dynamically.

Configuration files are an excellent example of a file that you don't want to bundle at build time because it's environment specific.

Instead, you can mount the configuration for a specific environment at runtime when you decide where the container should run.

Forwarding ports

Imagine you packaged a python web server as a Docker image.
The web server exposes port 8080, and if you visit that port, you can receive a response.

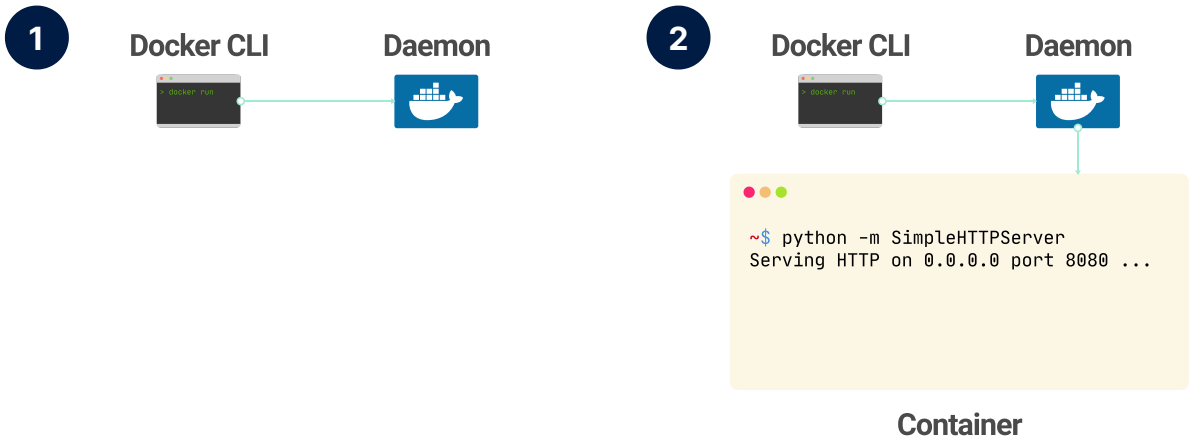


Fig. 1 You have a Docker client connected to a daemon.

Fig. 2 You create a container for a python web-server on port 8080.

You launch the container with `docker run` and open the browser on `http://localhost:8080`.
Do you expect to receive a response from the web server?



Fig. 1 You decide to visit the web server on `localhost:8080`

Fig. 2 But it doesn't work.

Unfortunately, the page times out.

Why?

When you create a container, Docker assigns an IP address and attaches the container to the default network.

The web server is not bound on localhost, but the container's IP address.

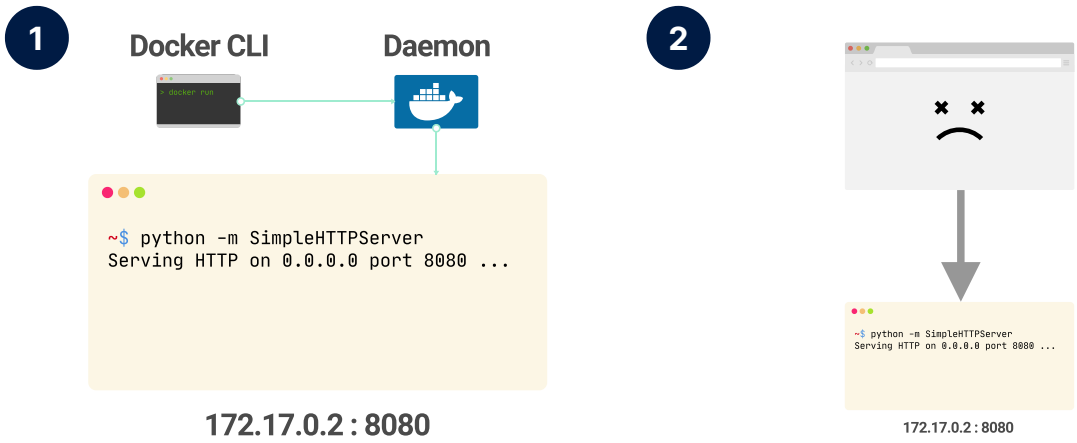


Fig. 1 When you run a container, the Docker daemon assigns an IP address and attaches it to the network.

Fig. 2 The web server is exposed on port 8080 on the IP address of the container. There's no process listening for port 8080 on your computer.

Since forwarding traffic to the container is frequently needed, Docker has a handy feature where you can send all the traffic from your localhost to the container.

When you launch your container with `docker run` you can use the `-p` flag — shorthand for *port*.

bash

```
$ docker run -p 8080:8080 python-web
```

Docker automatically creates port forwarding rules to route

the traffic from port 8080 on localhost to port 8080 inside the container.

You can also change the mapping and have port 3000 on your computer forward the traffic to port 8080 in the container:

```bash
$ docker run -p 3000:8080 python-web
```

You can bind any port from your container to your host **at run time**.

Managing configuration

We already discussed how using volumes is a convenient way to mount files and configuration at run time.

There's another way to pass arguments when a container is launched.

You can use the `-e` flag, *shorthand for environment*, to inject environment variables in the container.

```bash
$ docker run -ti -e HELLO=world bash
```

You can use the `-e` flag multiple times and inject several environment variables at once.

But how do you test if the environment variable is set up correctly?

Can you SSH inside the container and print it?

Attaching to running containers

Unlike virtual machines, you don't need to install SSH to open an interactive session inside the container.

Since containers are processes, you can attach to the running process directly.

You can use the `docker exec` command to start a session in the running container:

```bash
$ docker exec -ti <container id or name> bash
```

The command above opens a `bash` session inside the container.

Like if you were to use SSH.

As for the `docker run`, the `-ti` flags have a similar meaning:

- `-t` for terminal
- `-i` for interactive

> While easier to remember, the above should read: `-it` instructs Docker to allocate a pseudo-TTY connected to the container's stdin; creating an interactive bash shell in the container

You've learnt the basics of Linux containers and Docker. It's time to put your knowledge to practice.

Chapter 3

Installing Docker 

You can find the instructions to install Docker for you operating system below:

- Windows 10
- macOS
- Ubuntu

> Please note that only Windows 10 Pro is supported. If you need help with older versions of Windows, please head over to our detailed guide on how to install Docker and minikube on Windows.

Windows 10

You can download Docker for Windows with:

```
PowerShell                                                    — □ X

PS> choco install docker-desktop -y
Installing the following packages:
docker-desktop
By installing you accept licenses for the packages.
# truncated output
Installing docker-desktop...
docker-desktop has been installed.
```

You should be aware that Docker requires VT-X/AMD-v

virtual hardware extension to be enabled before you can run any container. Depending on your computer, you may need to reboot and enable it in your BIOS.

If you're unsure VT-X/AMD-v was enabled, don't worry. If you don't have it, Docker will greet you with the following error message:

> Hardware assisted virtualization and data execution protection must be enabled in the BIOS.

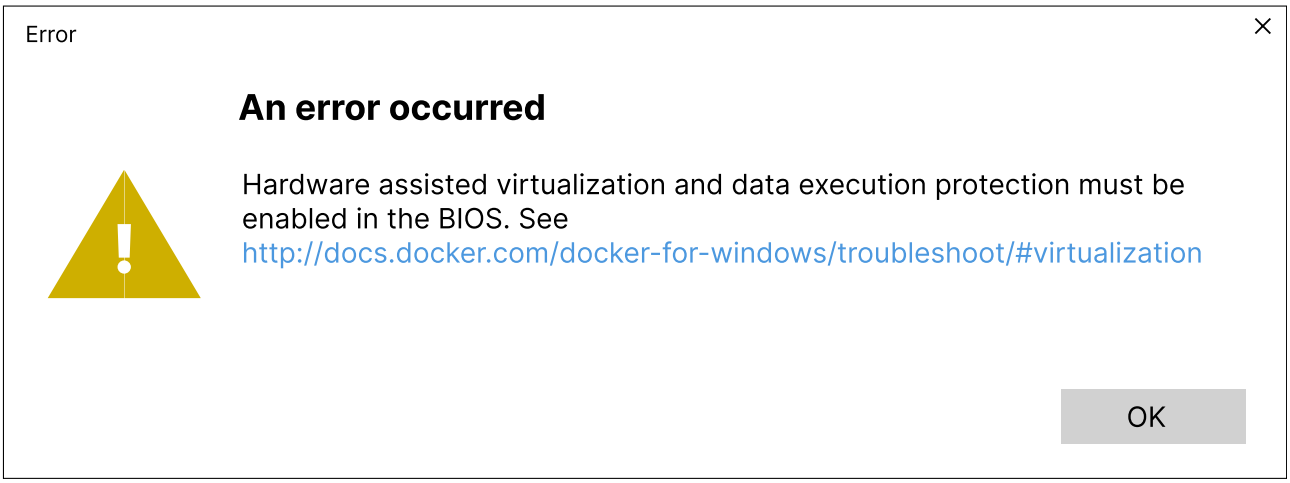


Fig. You should enable VT-X/AMD-v

Another common error has to do with the Hyper-V hypervisor not being enabled. If you experience this error:

> Unable to start: The running command stopped because the preference variable "ErrorActionPreference" or common parameter is set to Stop: 'MobyLinuxVM' failed to start. Failed to start the virtual machine 'MobyLinuxVM' because one of the Hyper-V components is not running.

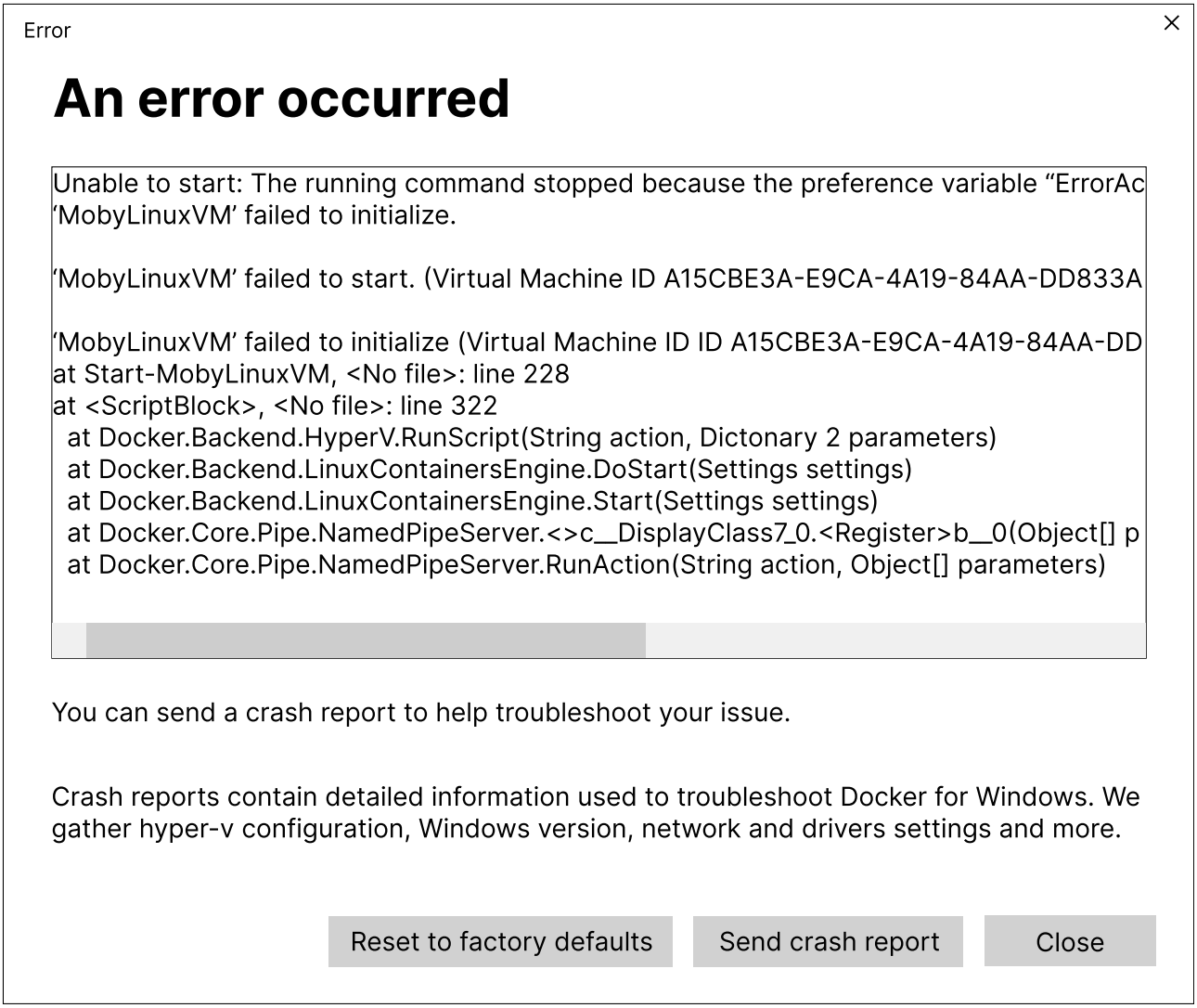


Fig. You should enable Hyper-V with Docker for Windows

You should enable Hyper-V.

Open a new command prompt as an administrator and type the following:



```
PS> bcdedit /set hypervisorlaunchtype auto
```

You should reboot your machine and Docker should finally start.

But how do you know if Docker is working?

Open a new command prompt and type:

```
PowerShell                                                    — □ X

PS> docker ps
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS   PORTS   NAMES
```

You can verify that Docker is installed correctly with the following command:

```
PowerShell                                                    — □ X

PS> docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

If your Docker daemon isn't running, you're probably banging your head against this error:

```
PowerShell                                                    — □ X

PS> docker ps
error during connect: Get http://%2F%2F.%2Fpipe%2Fdocker_engine/v1.37/containers/json:
open //./pipe/docker_engine: The system cannot find the file specified. In the default
daemon configuration on Windows, the docker client must be run elevated to connect. This
error may also indicate that the docker daemon is not running.
```

The error above is suggesting that your Docker installation is not behaving normally and wasn't able to start.
You should start your Docker daemon before you connect to it.

macOS

You can download Docker for Mac with:

```bash
$ brew install --cask docker
==> Downloading …
######################################################################## 100.0%
==> Installing Cask docker
==> Moving App 'Docker.app' to '/Applications/Docker.app'.
🍺  docker was successfully installed!
```

You should start Docker. It will ask permission to install the networking component.

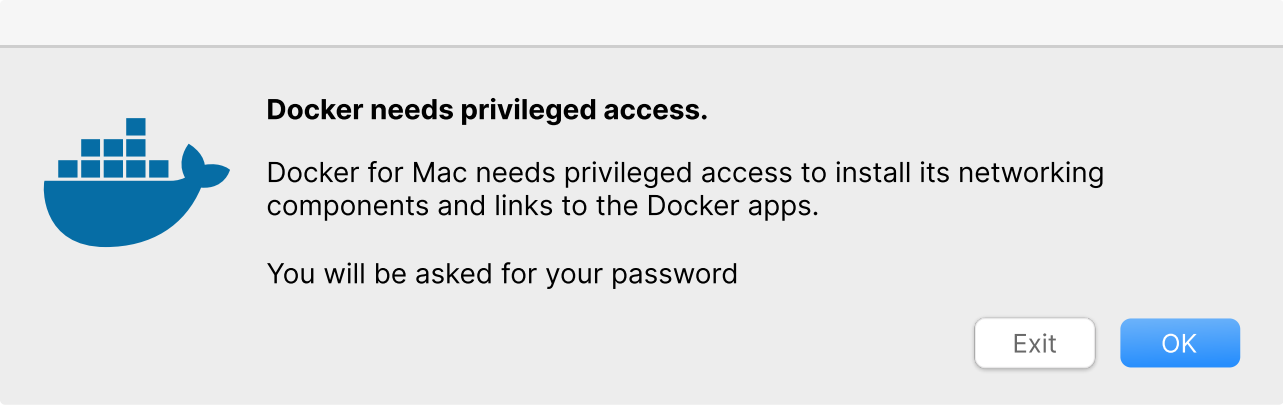


Fig. Docker networking component

You should proceed.
You can find Docker in your Applications folder.

Testing your Docker installation

You should test that your Docker installation was successful.
In your terminal type:

bash

```
$ docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Congratulations!
The installation was successful.

Ubuntu

Update the `apt` package index:

bash

```
$ sudo apt-get update -qq
```

Install packages to allow apt to use a repository over HTTPS:

bash

```
$ sudo apt-get install \
    apt-transport-https \
    ca-certificates \
    curl \
    gnupg-agent \
    software-properties-common
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
# truncated output
Do you want to continue? [Y/n] Y
done.
```

Add Docker's official GPG key:

bash

```
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

```
OK
```

Use the following command to add the stable repository.

bash

```
$ sudo add-apt-repository \
    "deb [arch=amd64] https://download.docker.com/linux/ubuntu \
    $(lsb_release -cs) \
    stable"
# truncated output
Reading package lists... Done
```

Update the apt package index again:

bash

```
$ sudo apt-get update -qq
```

Finally, install the latest version of Docker:

bash

```
$ sudo apt-get install -qqy docker-ce docker-ce-cli containerd.io
Unpacking docker-ce ...
Unpacking docker-ce-cli ...
Unpacking containerd.io ...
Setting up docker-ce ...
Setting up docker-ce-cli ...
Setting up containerd.io ...
```

You should test that your Docker installation was successful. In your terminal type:

```bash
$ docker run -ti -p 8080:80 nginx
```

You can verify that Docker is installed correctly with the following command:

```
PowerShell
PS> docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Congratulations!
The installation was successful.

Chapter 4

Getting started with Docker

Before you start, make sure that you have followed the instructions to install and test Docker on your computer.

bash

```
$ docker version
Client:
 Version:           20.10.6
 # output truncated

Server: Docker Engine - Community
 Engine:
  Version:          20.10.6
 # output truncated
```

Running Docker containers

You can start a Docker container with:

bash

```
$ docker run -ti python:3.9.5-slim-buster
Python 3.9.5 (default, May 27 2021, 10:05:38)
[GCC 8.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

You can break down the command in the following parts:

- `docker run` is the command to start a container.
- `-ti` is short for `-t` and `-i`. Those flags are often used

- together to tell docker to start an interactive session.
- `python` is the name of the image.
- `3.9.5-slim-buster` is the particular version of the image you wish to use.

If this is the first time that you're using that particular image, Docker will download the image from the registry.

You might notice that Docker downloads each individual image layer, one at a time:

```
bash

$ docker run -ti python:3.9.5-slim-buster
latest: Pulling from library/python
673aeee5c81c: Downloading [==>     ]   14.2MB/23.7MB
018b2790219d: Download complete
509c77ce92ad: Download complete
0cd458c20dd1: Waiting
fb1c02642395: Waiting
12d074f6952b: Waiting
```

You can now stop the Python interpreter within `Ctrl+D`, or you can type `quit()`.

Docker registries

The Python image that you just downloaded comes from Docker Hub.

Docker Hub is a public container registry.

You can create an account and upload your public images.

If you prefer to upload your images to a private registry, there're alternatives such as the premium version of Docker Hub, Bintray and Artifactory.

Also, each major cloud provider offers a private Docker registry:

- Amazon Web Services provides the Elastic Container Registry (ECR).
- Azure has the Azure Container Registry (ACR).
- Google Cloud Platform provides the Google Container Registry (GCR).

Running commands inside containers

The previous command ran the `python` executable by default.

If you wish to run another command, you can do so with:

bash

```
$ docker run -ti python:3.9.5-slim-buster bash
root@d97e62f6ac55:/#
```

Containers are excellent as a volatile environment where you can test your ideas.

Let's assume you want to install and try `figlet` — a binary that displays words as ASCII art.

You could launch an Ubuntu container with:

bash

```
$ docker run -ti ubuntu:20.10
root@d97e62f6ac55:/#
```

> Please notice how the default command for the Ubuntu image is `bash`.

You can install `figlet` with:

bash@ubuntu:20.10

```
$ apt-get update
$ apt-get install figlet
```

And you can test it with:

bash@ubuntu:20.10

```
$ figlet hello
 _          _ _
| |__   ___| | | ___
| '_ \ / _ \ | |/ _ \
| | | |  __/ | | (_) |
```

```
|_| |_|\___|_|_|\___/
```

You didn't have to install anything other than Docker to run the `figlet` command.
As soon as you type `exit` the interactive session is closed.
Where is the container gone?
Is it forever lost?

Start, stop, list, kill

You can list all running and stopped containers on your computer with:

bash

```
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND       CREATED        STATUS                     PORTS   NAMES
a11a7f4eaef4   ubuntu:20.10   "/bin/bash"   2 hours ago    Exited (0) 2 seconds ago           inspiring_beaver
```

> If you omit the `-a` flag, Docker will list only the running containers.

The container is still there, but it's stopped.
If you want to delete a stopped container, perhaps because

you want to reclaim the space, you can do so with:

bash

```
$ docker rm <container_id>
```

If you wish to start a container that is currently stopped, you can do so with:

bash

```
$ docker start -ai <container_id>
```

> The `-ai` flag is used to attach the input and output to your terminal session.

Starting stopped containers isn't a very popular task.
It's usually quicker to create a new container instead.
If you want to remove a container automatically after it has exited, you can do so with:

bash

```
$ docker run -ti --rm ubuntu:20.10
root@d97e62f6ac55:/#
```

The `--rm` flag instructs Docker to remove the container once the process exits.

There's another `docker run` flag that you might often find yourself using: the `-d` flag.

You can use the `-d` flag to launch a container in the background.

As an example, the following container displays the time and is always in the foreground:

bash

```
$ docker run -ti jpetazzo/clock
Wed Jun  2 08:09:45 UTC 2021
Wed Jun  2 08:09:46 UTC 2021
Wed Jun  2 08:09:47 UTC 2021
Wed Jun  2 08:09:48 UTC 2021
^C
```

But you can start the same container in the background, as a deamon, with the `-d` flag:

bash

```
$ docker run -d jpetazzo/clock
57350cb197a3ce1686e03fc5d2311a41bb00cf8761aa7465714236897f5b37fe
```

The command returns a long hash.

Where is the output gone, though?

You can list all the running containers with:

bash

```
$ docker ps
```

```
CONTAINER ID   IMAGE             COMMAND                  CREATED          STATUS
57350cb197a3   jpetazzo/clock    "/bin/sh -c 'while d…"   38 seconds ago   Up 37 seconds
```

You can tail the logs of the container running as a deamon with:

bash

```
$ docker logs <container id>
Wed Jun  2 08:09:45 UTC 2021
Wed Jun  2 08:09:46 UTC 2021
Wed Jun  2 08:09:47 UTC 2021
Wed Jun  2 08:09:48 UTC 2021
```

Once you're satisfied, you can stop a running container with:

bash

```
$ docker stop <container id>
```

The `docker stop` command sends the `SIGTERM` signal to the process and waits for it to terminate gracefully.

If you don't mind terminating a process abruptly, you can force stopping a running container with:

bash

```
$ docker kill <container id>
```

`docker kill` sends the `SIGKILL` signal to the process.

Terminology

In Docker, it's easy to confuse the terms containers and images.

- **Containers** are created from images. You can have multiple containers instantiated from the same image.
- **Images** are similar to templates. When you want to run a container, you select and copy the template to run it.

An image can have multiple containers.
However, a container can only be created from a single image.
If you are still confused, you can think about images as Java classes.
You can't run a Java class by itself.
You should be instantiating it first.
An instance of an image is a container.

Recap

- You can run a container with `docker run`.
- There are three valuable flags for `docker run`:
  1. `-ti` to attach an interactive session.
  2. `-d` to launch the container in the background.
  3. `--rm` to remove the container when the process exits.
- You can list containers with `docker ps`.
- You can stop containers with `docker stop` and force stopping them with `docker kill`.

Chapter 5

Creating images

When you execute `docker run`, there are two crucial bits of information that you have to specify:

1. The Docker image and
2. the process.

As an example, `docker run ubuntu bash` runs the `bash` process inside the Ubuntu file system.

But weren't containers supposed to share the same kernel as the operating system?

Why do you need an entire operating system on top of the existing operating system to run a container?

When you start a container, the image is used to provide an isolated file system to the process.

You can select Ubuntu as the file system, or you could offer no files at all (the Ubuntu image starts with no files at all).

Why do you need to provide files to the process?

The process might required shared libraries such as `glibc` or `muslc` to function correctly.

Most of the times, you will find yourself customising the base image and adding:

1. Specify dependencies such as tools and libraries.
2. Your application's files.

You can build your own Docker images by writing a `Dockerfile` — a text file that contains all the commands

that should be executed to create the image.

Your first Dockerfile

Let's assume you want to create a simple image that includes the `figlet` binary.
Let's start with creating a temporary directory:

bash

```
$ mkdir first-dockerfile
```

You should create a `Dockerfile` inside it:

bash

```
$ cd first-dockerfile
$ code Dockerfile
```

> Please note the capital D in `Dockerfile`. Also, the file has no extension.

The content should be:

Dockerfile

```
FROM ubuntu:20.10
RUN apt-get update
RUN apt-get -y install figlet
```

The `Dockerfile` contains two instructions:

- `FROM`. Your image will inherit from the Ubuntu image,
- `RUN` executes a command and create an image layer.

Building the image

You can build your first image with:

bash

```
$ docker build -t first .
[+] Building 11.9s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 106B
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load metadata for docker.io/library/ubuntu:20.10
 => [1/3] FROM docker.io/library/ubuntu:20.10
 => [2/3] RUN apt-get update
 => [3/3] RUN apt-get -y install figlet
 => exporting to image
 => => exporting layers
 => => writing image sha256:6f4fcea87e54d2d8c76f5606b5a80ffaaa04c196071bb9642dd1c9bb40e27669
 => => naming to docker.io/library/first
```

That's a long output!

What did just happened?

Let's have a look at the command first:

- `docker build` is the command to build an image.
- `-t first` is the name you wish to give to the image. `-t` is a short name for `--tag`.
- `.` is the context directory. It's common practice to have the `Dockerfile` in the root of this directory.

As soon as you type the command, Docker executes a few steps.

The first step is uploading the current context to the Docker deamon.

In this phase, the Docker CLI check if a `.dockerignore` file contains a list of files that should be excluded from the context.

bash

```
$ docker build -t first .
[+] Building 11.9s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 106B
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load metadata for docker.io/library/ubuntu:20.10
 => [1/3] FROM docker.io/library/ubuntu:20.10
 => [2/3] RUN apt-get update
 => [3/3] RUN apt-get -y install figlet
 => exporting to image
 => => exporting layers
 => => writing image sha256:6f4fcea87e54d2d8c76f5606b5a80ffaaa04c196071bb9642dd1c9bb40e27669
 => => naming to docker.io/library/first
```

In the next few steps, the Docker deamon reads the instructions from the `Dockerfile` and executes them one at a time.

bash

```
$ docker build -t first .
[+] Building 11.9s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 106B
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load metadata for docker.io/library/ubuntu:20.10
 => [1/3] FROM docker.io/library/ubuntu:20.10
 => [2/3] RUN apt-get update
 => [3/3] RUN apt-get -y install figlet
 => exporting to image
 => => exporting layers
 => => writing image sha256:6f4fcea87e54d2d8c76f5606b5a80ffaaa04c196071bb9642dd1c9bb40e27669
 => => naming to docker.io/library/first
```

The remaining instructions save the generated layers on the file system and compute the hash for the container.

bash

```
$ docker build -t first .
[+] Building 11.9s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 106B
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load metadata for docker.io/library/ubuntu:20.10
 => [1/3] FROM docker.io/library/ubuntu:20.10
 => [2/3] RUN apt-get update
 => [3/3] RUN apt-get -y install figlet
 => exporting to image
 => => exporting layers
 => => writing image sha256:6f4fcea87e54d2d8c76f5606b5a80ffaaa04c196071bb9642dd1c9bb40e27669
 => => naming to docker.io/library/first
```

You can verify that the image was built correctly with:

bash

```
$ docker images | grep first
REPOSITORY     TAG       IMAGE ID       CREATED          SIZE
first          latest    6f4fcea87e54   9 minutes ago    105MB
```

Finally, you can run the image with:

```bash
$ docker run -ti first
root@b86e0997b1c9:/#
```

Is the `figlet` *binary available?*

You can test it with:

```bash
$ figlet "learnk8s"
 _                                 _    ___
| | ___  __ _ _ __ _ __ | | _( _ ) ___
| |/ _ \/ _` | '__| '_ \| |/ / _ \/ __|
| |  __/ (_| | |  | | | |   < (_) \__ \
|_|\___|\__,_|_|  |_| |_|_|\_\___/|___/
```

Excellent!

Notice how you didn't install the `figlet` binary because it was already bundled in the image.

Recap

- Docker images are a collection of files needed by the process to run.
- When the container starts, you can execute any binary inside that image.

- You can use a prebuilt image like Ubuntu, or you can build your own.
- The root image is the empty image (also called `SCRATCH`).
- You can build your Docker images by defining a `Dockerfile`.
- A `Dockerfile` is a list of instructions executed by the Docker deamon.

Chapter 6

Managing state

Containers are isolated processes that share the kernel with their host.

But most of the time, you want to share more than the kernel.

A common use case is sharing files with the container.

Sharing the file system

On your computer, create a folder and a dummy file:

bash

```
$ mkdir shared
$ echo "Hello World" > shared/test.txt
```

Mount the directory `./shared` as `/myfolder` inside the container:

bash

```
$ docker run -v ${PWD}/shared:/myfolder -ti busybox
/ #
```

> Please notice that if you see an error at this point, you might need to grant permission to Docker to use that folder. Open Docker and check the **Settings** for **"File Sharing"**.

The `-v` flag is used to map a local folder on the host.
In this case, the folder `./shared` on your computer is mapped to `/myfolder` inside the container.
Docker accepts only absolute paths as arguments for the `-v` flag.
The `${PWD}` environment variable is a convenient way to provide the full path of that current directory.
Once you're inside the container, you should check that the content of the file is identical:

sh@busybox

```
$ cat /myfolder/test.txt
Hello World
```

Excellent!
What happens when you write in the mounted directory?

Exploring bidirectional sync

From within the container create a file:

sh@busybox

```
$ echo "Hello from the container!" > /myfolder/intro.txt
$ exit
```

If you exit from the container, you should notice that the same file is now located on your host:

bash

```
$ cat shared/intro.txt
Hello from the container!
```

The directory was shared between the host and the container.

Including files in the container image

You can only mount volumes when you run a container.

But how do you copy your jars, executables or scripts inside the image during the build phase?

You can use the `COPY` command in your `Dockerfile` to copy files from your computer into the container image.

When the container starts, those files are available to the process.

Let's explore the `COPY` *command.*

Create a `Dockerfile` in the shared directory as follow:

bash

```
$ code shared/Dockerfile
```

And save the following content:

Dockerfile

```
FROM busybox
COPY ./test.txt /myfolder/test.txt
```

You can build your image with:

bash

```
$ cd shared
$ docker build -t built-in .
[+] Building 0.1s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 87B
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load metadata for docker.io/library/busybox:latest
 => [1/2] FROM docker.io/library/busybox
 => [internal] load build context
 => => transferring context: 38B
 => [2/2] COPY ./test.txt /myfolder/test.txt
 => exporting to image
 => => exporting layers
 => => writing image sha256:48e75691e9a7a68ce54df24393d6ede94dbcd1b31b24a4787d1d79899f7379cb
 => => naming to docker.io/library/built-in
```

You can test the image with:

bash

```
$ docker run -ti built-in
/ #
```

As soon as you are inside the container, you can retrieve the content of the file:

sh@busybox

```
$ cat /myfolder/test.txt
Hello World
```

What happens when you edit the file?
Does it change on the host too?
You can overwrite the file from within the container with:

sh@busybox

```
$ echo "Edited" > /myfolder/test.txt
```

If you exit from the container and retrieve the file from the host, you will notice that the file hasn't changed:

bash

```
$ cat test.txt
Hello World
```

The file was changed in the file system of the container.

You did not share any volume between the host and container.

Recap

- You can use a Docker volume to share a directory from the host with the container.
- Docker volumes are created at runtime and are helpful to share configuration, secrets or data with the container.
- If you want to bundle a file with a container image, you should use the `COPY` command in your `Dockerfile`.
- Files copies in the container are not kept in sync with the host.

Chapter 7

Networking

Docker containers have IP addresses assigned to them.

By default, they are also attached to a bridge network and can talk to each other.

Docker containers can consume services from the internet too.

But can you connect to a Docker container?

Connecting to the container

On Docker Hub, there's an image for Nginx — a popular web server.

Nginx is usually used as a reverse proxy, a web server or a load balancer.

In the rest of this chapter, you will use it as a simple webserver to display a *Hello World* web page.

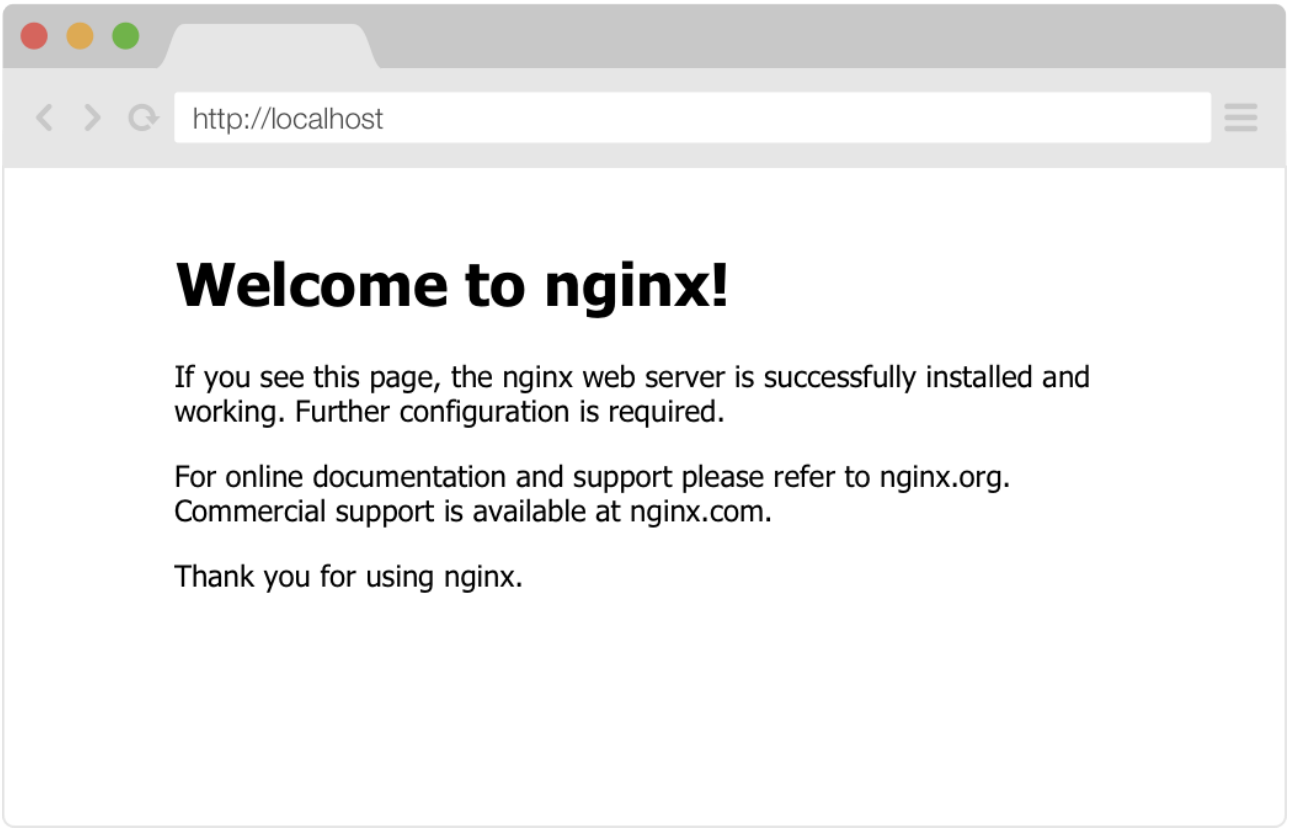


Fig. Nginx Hello World

You can start the container with:

```bash
$ docker run -d nginx:1.21.0-alpine
8770f19754ec0c30cb004
```

The container is launched with the `-d' flag and starts in the background.

By default, Nginx binds on port 80.

What do you expect to see when you visit http://localhost?
Are you greeted by the Nginx "Welcome to nginx!" page?
Unfortunately, no.
The port is opened on the container, but you're visiting localhost on port 80 on your computer.
How do you visit port 80 on the container itself?
Before proceeding any further, you should stop the current container.
You can list the running containers with:

bash

```
$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED         STATUS         PORTS
8770f19754ec   nginx:1.21.0-alpine   "/docker-entrypoint.…"   2 minutes ago   Up 2 minutes   80/tcp
```

And you can stop it with:

bash

```
$ docker stop <container_id>
```

Binding container ports to the host

You can set up a port forwarding rule when you launch the container with `docker run`:

```bash
$ docker run -d -p 8080:80 nginx:1.21.0-alpine
8770f19754ec0c30cb004
```

The `-p` flag instructs Docker to bind the port 8080 on your host and forward the traffic to port 80 on the container.

You can verify the bindings with:

```bash
$ docker ps
CONTAINER ID   IMAGE                COMMAND                  PORTS
aed1a13d6ba9   nginx:1.21.0-alpine  "/docker-entrypoint.…"   0.0.0.0:8080->80/tcp, :::8080->80/tcp
```

The `PORTS` column suggests that the application is bound correctly.

You can visit your application at http://localhost:8080 — this time, it should work.

> Note how the application still thinks it's running on port 80.

Recap

- Docker containers are assigned an IP address.

- All Docker containers join the default bridge network and can talk to each other.
- When a container exposes a web server, the process binds the port on the container.
- You can set up forwarding rules to route the local traffic to the container.

Chapter 8

Packaging an application

Let's recap some of the fundamental concepts when using containers with a practical example.

Hello World, Node.js

In this section, you will create a simple *Hello World* application written in Node.js.

There's no need to install Node.js on your computer: you will install all the dependencies in the container.

Create a new directory:

bash

```
$ mkdir -p docker-hello-world
$ cd docker-hello-world
```

Inside that directory, create an `index.js` file with the following content:

index.js

```
const http = require('http')

const port = 8080
const version = process.env.VERSION || 'unknown'

const server = http.createServer((req, res) => {
  res.statusCode = 200
```

```
  res.setHeader('Content-Type', 'text/plain')
  res.end(`Hello World\nVersion: ${version}\n`)
});

server.listen(port, () => {
  console.log(`Server running at http://localhost:${port}/`)
})
```

The application setups a single HTTP handler: every time there is a request, it replies with the string `Hello world` and the current `VERSION` of the app.
The `VERSION` is an environment variable that you can set.
If you don't set one, the application replies with `unknown` as a version.

Creating an image

You should create a `Dockerfile` to package your app.

bash

```
$ code Dockerfile
```

It should have the following content:

Dockerfile

```
FROM node:16-slim
WORKDIR /app
COPY . .
EXPOSE 8080
CMD ["node", "index.js"]
```

Take your time to inspect the `Dockerfile`.

- `FROM node:16-slim` is the base container image.
- `WORKDIR /app` sets the current directory inside the container to be `/app`.
- `COPY . .` copies all the files from the host in the current directory (since you changed it in the previous step, the directory is `/app`).
- `EXPOSE 8080`. This is a comment. It doesn't expose any port, but it's useful as documentation.
- `CMD ["node", "index.js"]`, this is default command. When the container starts, it executes the `node index.js` command.

You can create the image with:

bash

```
$ docker build -t hello-world .
+] Building 9.3s (8/8) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 116B
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load metadata for docker.io/library/node:16-slim
 => [1/3] FROM docker.io/library/node:16-slim
 => [internal] load build context
 => => transferring context: 543B
 => [2/3] WORKDIR /app
 => [3/3] COPY . .
 => exporting to image
```

```
=> => exporting layers
=> => writing image sha256:5d1559f67831795d069f8501b3b7702476bd3bae45d8eb4698b188c7df7833b8
=> => naming to docker.io/library/hello-world
```

You can run the application with:

bash

```
$ docker run -ti -p 8080:8080 hello-world
Server running at http://127.0.0.1:8080/
```

The container executes the default command and starts the webserver.

You can visit the application in your browser at http://localhost:8080, or you can issue a `curl` request:

bash

```
$ curl http://localhost:8080
$ Hello World
Version: unknown
```

Can you inject a version in the container?

Yes, but not at runtime.

So you should stop the container first.

In a new terminal type:

bash

```
$ docker ps
CONTAINER ID   IMAGE         COMMAND
```

```
 PORTS
d8827ff35ab6   hello-world   "docker-entrypoint.s…"   0.0.0.0:8080->8080/tcp
```

You can stop the container with:

bash

```
$ docker stop <container_id>
```

Injecting environment variables

If you wish to inject a `VERSION` environment variable into your container, you can do so with:

bash

```
$ docker run -ti -p 8080:8080 -e VERSION=1 hello-world
Server running at http://127.0.0.1:8080/
```

> Please note that you can inject multiple environment variables by adding the flag `-e` multiple times.

You can verify that the version was successfully updated with:

```bash
$ curl http://localhost:8080/
Hello World
Version: 1
```

Injecting environment variables in containers is convenient to decouple configuration from the application.

You should not copy the configuration in your container with the `COPY` command.

You should either:

1. Mount a volume with the `-v` flag or
2. Inject environment variables with the `-e` flag.

If you follow those best practices, you can use the same container images for your development, preproduction and production environments.

Recap

In this chapter, you managed to package an application as a container.

- You don't have to install Node.js to package the application as a container.

- You can create a container image with a `Dockerfile`.
- In the `Dockerfile`:
  - The `WORKDIR` command sets the current directory. All subsequent instructions use that as the base directory.
  - The `EXPOSE` command isn't a real command. It's a comment, and removing it doesn't affect what port is exposed by the container.
  - The `CMD` command sets the default command. When the container starts, it executes that command.
- You can inject the environment variable in the container with the `-e` flag.
- If you need to inject multiple environment variables, you can use the `-e` flag multiple times.
- You should use environment variables or mount volumes to pass configurations or secrets to the app.

Chapter 9

Troubleshooting

When building and running containers, you may encounter two kinds of errors:

1. **Errors during the build phase.** Those are usually due to wrong commands in the Dockerfile.
2. **Runtime errors** due to the application misbehaving.

Let's have a look at how you can troubleshoot them.

Attaching to a running container

If you think there's an issue with your application, you can attach an interactive session to the current container.

The procedure is very similar to SSH'ing into a virtual machine.

Let's start by launching an Ubuntu container with few environment variables:

bash

```
$ docker run -ti -e ENV=TEST ubuntu:20.10
root@f6e607798004:/#
```

In another terminal, you can retrieve the id of the container with:

bash

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND
f6e607798004   ubuntu:20.10   "/bin/bash"
```

You can attach to a running container with:

bash

```
$ docker exec -ti <container_id> bash
root@f6e607798004:/#
```

To inspect the value of `ENV` you can execute the following command from within the container:

bash@container

```
$ echo $ENV
TEST
```

When you attached to the running container, you ran the `bash` binary.
But you could also run other commands such as `whoami`:

bash

```
$ docker exec <container_id> whoami
root
```

The command is executed inline, and the output returned

immediately.

Attaching to a container is helpful to troubleshoot issues with your application.

Common challenges when attaching to containers

When running or attaching to running containers, you may not be able to execute the `bash` binary:

bash

```
$ docker run -ti busybox bash
docker: Error response from daemon: OCI runtime create failed: container_linux.go:367:
starting container process caused: exec: "bash": executable file not found in $PATH: unknown.
ERRO[0000] error waiting for container: context canceled
```

The command fails.

Indeed, there's no `bash` in busybox.

But there's `sh`:

bash

```
$ docker run -ti busybox sh
/ #
```

In other words, you can only execute binaries that are part of

your image.

Troubleshooting build errors

When building containers with `docker build`, it's common to make mistakes.

Consider the following `Dockerfile` with a subtle mistake:

Dockerfile

```
FROM ubuntu:20.10
RUN mkdir /temp
RUN apt-get -y install figlet
```

If you try to build the container, you might see this:

bash

```
$ docker build -t error .
[+] Building 0.3s (6/6) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 34B
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load metadata for docker.io/library/ubuntu:20.10
 => [1/3] FROM docker.io/library/ubuntu:20.10
 => CACHED [2/3] RUN mkdir /temp
 => ERROR [3/3] RUN apt-get -y install figlet
------
 > [3/3] RUN apt-get -y install figlet:
#5 0.173 Reading package lists...
#5 0.191 Building dependency tree...
#5 0.191 Reading state information...
#5 0.192 E: Unable to locate package figlet
------
executor failed running [/bin/sh -c apt-get -y install figlet]: exit code: 100
```

Can you spot the error?

If you can't, it'd be great to launch the container at the last working step and execute the rest of the commands manually.

You can print the intermediate layers with:

INSTRUCTIONS FOR BASH

bash

```
$ DOCKER_BUILDKIT=0 docker build -t error .
Sending build context to Docker daemon  2.048kB
Step 1/3 : FROM ubuntu:20.10
 ---> 616e686db0f3
Step 2/3 : RUN mkdir /temp
 ---> Running in fb7fa1bbf3dd
Removing intermediate container fb7fa1bbf3dd
 ---> e6420365fd11
Step 3/3 : RUN apt-get -y install figlet
 ---> Running in dc6cee9b928c
Reading package lists...
Building dependency tree...
Reading state information...
E: Unable to locate package figlet
The command '/bin/sh -c apt-get -y install figlet' returned a non-zero code: 100
```

INSTRUCTIONS FOR POWERSHELL

PowerShell — ◻ X

```
PS> $Env:DOCKER_BUILDKIT=0; docker build -t error .
Sending build context to Docker daemon  2.048kB
Step 1/3 : FROM ubuntu:20.10
 ---> 616e686db0f3
Step 2/3 : RUN mkdir /temp
 ---> Running in fb7fa1bbf3dd
Removing intermediate container fb7fa1bbf3dd
 ---> e6420365fd11
Step 3/3 : RUN apt-get -y install figlet
 ---> Running in dc6cee9b928c
Reading package lists...
Building dependency tree...
Reading state information...
E: Unable to locate package figlet
The command '/bin/sh -c apt-get -y install figlet' returned a non-zero code: 100
```

You can run the layer before the error with:

bash

```
$ docker run -ti --rm e6420365fd11 bash
root@1b573966b5b3:/#
```

At this point, you are ready to debug the issue.
Let's test the same command:

bash

```
$ apt-get -y install figlet
Reading package lists... Done
Building dependency tree
Reading state information... Done
E: Unable to locate package figlet
```

Perhaps, you are missing an `apt-get update`?

bash

```
$ apt-get update && apt-get -y install figlet
Reading package lists... Done
Building dependency tree
Reading state information... Done
E: Unable to locate package figlet
root@1b573966b5b3:/# apt-get update && apt-get -y install figlet
Get:1 http://ports.ubuntu.com/ubuntu-ports groovy InRelease [267 kB]
Get:2 http://ports.ubuntu.com/ubuntu-ports groovy-updates InRelease [115 kB]
# truncated
Unpacking figlet (2.2.5-3) ...
Setting up figlet (2.2.5-3) ...
```

Hurrah!
You can now fix the `Dockerfile`:

Dockerfile

```
FROM ubuntu:20.10
RUN mkdir /temp
RUN apt-get update
RUN apt-get -y install figlet
```

Recap

- You can attach to a running container with `docker exec`.
- If you want to connect to a container, but the image doesn't contain `bash`, you should try with another binary (perhaps `sh`).
- If you see an error during the build phase for your container image, you can start an intermediate container and inspect the filesystem.